数据结构与算法知识库文档

第一章 绪论

- 数据结构基本概念：
- 数据相关定义：
- 数据：不仅是客观事物的符号表示，还包括能够被计算机识别、存储和处理的一切信息。例如，数值、字符、图像、音频、视频等都属于数据范畴。
- 数据元素：作为数据的基本单位，在不同的场景下有不同的称呼。比如，在学生信息表中，每个学生的信息就是一个数据元素；在图书目录中，每本图书的记录也是一个数据元素。它是进行数据处理的基本单位，由若干个数据项组成。
- 数据项：是构成数据元素的不可分割的最小单位，用于描述数据元素的某个具体属性。例如，学生信息这个数据元素由姓名、学号、性别、年龄等数据项组成，每个数据项都不能再分割为更小的单位来描述学生信息的某个属性。
- 数据对象：是性质相同的数据元素的集合，是数据的一个子集。这里的“性质相同”指的是数据元素具有相同数量和类型的数据项。例如，全体学生的信息构成一个数据对象，因为每个学生信息这个数据元素都包含姓名、学号等相同类型和数量的数据项；所有整数也构成一个数据对象。
- 数据结构定义：
  数据结构是相互之间存在一种或多种特定关系的数据元素的集合，它主要研究数据的逻辑结构、存储结构以及对数据的运算。
- 逻辑结构：指数据元素之间的相互关系，与数据的存储位置无关。常见的逻辑结构有：
- 集合结构：数据元素之间除了同属于一个集合外，没有其他任何关系。例如，一个班级的所有学生构成一个集合，学生之间没有特定的顺序或层次关系。
- 线性结构：数据元素之间存在一对一的关系。除了第一个和最后一个元素外，其他每个元素都有且仅有一个前驱和一个后继。比如，线性表、栈、队列等都属于线性结构。
- 树结构：数据元素之间存在一对多的关系。有一个根节点，根节点之下有若干个子节点，每个子节点又可以有自己的子节点，以此类推。例如，家族树、组织结构树等都是树结构的具体体现。
- 图结构：数据元素之间存在多对多的关系。每个数据元素都可以与其他多个数据元素相关联。像城市交通图中，每个城市（数据元素）都可以与多个其他城市有道路连接（关系）。
- 存储结构：又称物理结构，是数据元素及其关系在计算机存储器中的表示。常用的存储结构有：
- 顺序存储结构：把数据元素存放在连续的存储单元中，数据元素之间的逻辑关系通过它们的物理位置（存储地址）来体现。例如，数组就是采用顺序存储结构，数组元素按照索引的顺序依次存储在连续的内存空间中。
- 链式存储结构：数据元素可以存放在不连续的存储单元中，通过指针来表示数据元素之间的逻辑关系。每个数据元素（节点）除了存储自身信息外，还包含一个或多个指针，指向其前驱或后继节点的存储地址。链表就是典型的链式存储结构。

- 索引存储结构：在存储数据元素的同时，还建立一个索引表。索引表中的每一项称为索引项，索引项一般由关键字和该关键字对应的数据元素的存储地址组成。通过索引表可以快速找到数据元素的存储位置，提高查找效率。例如，图书的目录就是一种索引，通过目录中的页码可以快速找到对应的内容。
- 散列存储结构：根据数据元素的关键字，通过散列函数计算出该数据元素的存储地址，并将其存放在该地址中。这种存储结构的目的是为了实现快速查找，哈希表就是采用散列存储结构。
- 抽象数据类型（ADT）：
  抽象数据类型是由用户定义的，表示现实世界中的某个问题的数学模型以及一组针对该模型的操作。它的定义只取决于其逻辑特性，而与计算机内部如何表示和实现无关。抽象数据类型可以看作是对数据的一种封装，将数据和操作数据的方法捆绑在一起，隐藏了数据的内部实现细节，只向外界提供使用接口。例如，栈就是一种抽象数据类型，它定义了入栈、出栈、判空等操作，无论栈是用数组（顺序存储）还是链表（链式存储）实现的，用户使用的都是这些相同的操作接口。
- 算法和算法分析：
- 算法定义与特性：
  算法是对特定问题求解步骤的一种描述，是指令的有限序列。它具有以下五个重要特性：
- 有穷性：算法必须在执行有限步之后结束，且每一步都在有限的时间内完成。例如，计算 1 到 100 的和，只需要执行 100 次加法操作就会结束，满足有穷性；而一个无限循环的程序就不能称为算法，因为它永远不会结束。
- 确定性：算法中的每一条指令都必须有确切的含义，不会产生二义性。对于相同的输入，算法必须能得到相同的输出。例如，“将 a 和 b 相加”是确定的指令，而“将 a 和 b 进行某种运算”就是不确定的，因为“某种运算”没有明确的定义。
- 可行性：算法描述的操作都可以通过已经实现的基本运算执行有限次来完成。例如，算法中涉及的加法、减法等运算都是计算机可以实现的基本运算。
- 输入：算法可以有零个或多个输入，这些输入是从特定的对象集合中选取的。例如，计算两个数的乘积，需要输入这两个数，这是有两个输入的情况；而输出“Hello World!”的算法则不需要输入。
- 输出：算法必须有一个或多个输出，输出的结果与输入有特定的关系。例如，排序算法的输入是一个无序的序列，输出则是一个有序的序列，这个有序序列就是对输入序列进行排序后的结果。
- 算法评价：
  评价一个算法的优劣通常从以下几个方面考虑：
- 正确性：算法能够正确地解决所求解的问题。即对于一切合法的输入数据，算法都能得出正确的输出结果。例如，一个排序算法能够将任意输入的无序序列按照规定的顺序排列好，就是正确的算法。
- 可读性：算法的描述应该清晰、易懂，便于人们理解和交流。可读性好的算法有利于后期的维护和修改。例如，使用结构化的编程语言、添加必要的注释等都可以提高算法的可读性。
- 健壮性：当输入的数据不合法时，算法能够做出相应的处理，而不是出现异常或崩溃。例如，当一个除法算法的输入中除数为零时，算法能够提示“除数不能为零”，而不是直接导致程序出错，就是健壮性好的表现。
- 高效性：包括时间效率和空间效率。时间效率指算法执行所需的时间，空间效率指

算法执行过程中所需的存储空间。高效的算法应该在保证正确性的前提下，尽可能地减少时间和空间的消耗。

- 复杂度衡量：
  算法的复杂度是衡量算法效率的重要指标，主要包括时间复杂度和空间复杂度。
- 时间复杂度：是指算法执行过程中所需基本操作的次数，它是问题规模 n 的函数，记为 T(n)。通常使用大 O 表示法来描述时间复杂度，即 $T(n)=O(f(n))$，其中 f(n) 是问题规模 n 的某个函数。大 O 表示法关注的是当 n 趋近于无穷大时，T(n) 的数量级。例如，一个算法的基本操作次数是 $3n^2+2n+1$，那么它的时间复杂度就是 $O(n^2)$。常见的时间复杂度按量级递增排列依次为：O(1)（常数阶）、O(log n)（对数阶）、O(n)（线性阶）、O(n log n)（线性对数阶）、$O(n^2)$（平方阶）、$O(n^3)$（立方阶）、$O(2^n)$（指数阶）等，其中 O(1) 效率最高，$O(2^n)$ 效率最低。
- 空间复杂度：是指算法在执行过程中所需临时占用的存储空间的大小，也是问题规模 n 的函数，记为 S(n)，通常也用大 O 表示法来描述，即 $S(n)=O(g(n))$。如果算法所需的临时存储空间不随问题规模 n 的变化而变化，即空间复杂度为 O(1)，则称该算法是原地工作的。例如，一个算法只需要使用几个固定的变量来存储中间结果，那么它的空间复杂度就是 O(1)；而一个需要动态分配大小为 n 的数组的算法，其空间复杂度就是 O(n)。

第二章 线性表

- 线性表定义和特点：
  线性表是由 n 个数据特性相同的元素构成的有限序列，通常表示为 L=(a_1，a_2，…，a_n)。其中，n 为线性表的长度，当 n=0 时，称为空表。
  非空线性表具有以下特点：
- 有唯一的第一个元素，即 a_1；有唯一的最后一个元素，即 a_n。
- 除第一个元素外，每个元素 a_i（i>1）有且仅有一个直接前驱元素 a_{i-1}；除最后一个元素外，每个元素 a_i（i<n）有且仅有一个直接后继元素 a_{i+1}。
  例如，英文字母表（A，B，C，…，Z）是一个线性表，其中每个字母是一个元素，它们之间存在着严格的先后顺序；学生成绩表中，按学号顺序排列的每个学生的成绩记录也构成一个线性表。
- 线性表顺序表示：
  线性表的顺序表示又称为顺序存储结构或顺序表，它是用一组地址连续的存储单元依次存储线性表中的各个元素。
  其特点是：逻辑上相邻的元素（如 a_i和 a_{i+1}）在物理位置上也相邻。
  假设线性表中每个元素占用 k 个存储单元，且第一个元素 a_1的存储地址为 LOC (a_1)，则元素 a_i的存储地址为 LOC (a_i)=LOC (a_1)+(i-1)×k。
  这种存储方式使得顺序表成为一种随机存取结构，即可以通过元素的索引（位置）直接计算出该元素的存储地址，从而快速访问该元素。例如，在数组中，通过下标可以直接访问对应的元素，就是利用了顺序表的随机存取特性。
- 顺序表操作：
- 取值：
  取值操作是指获取顺序表中第 i 个元素的值。由于顺序表是随机存取结构，根据元素的索引 i 可以直接计算出其存储地址，因此只需一次访问操作即可完成取值，时间复杂度为 O (1)。
  具体步骤：判断 i 的取值是否合法（1≤i≤n），若合法，则返回 LOC (a_1)+(i-1)×k 地址处的元素值；若不合法，则返回错误信息。
- 查找：
  查找操作是指在顺序表中查找值为 x 的元素，若找到，则返回该元素的位置索引；若未找到，则返回 0 或错误信息。
  查找过程从第一个元素开始，依次将每个元素与 x 进行比较，直到找到相等的元素或遍历完整个顺序表。
  最坏情况下，需要比较 n 次（x 是最后一个元素或不存在），时间复杂度为 O (n)。

  平均查找长度 ASL（Average Search Length）是指在查找过程中，平均比较的元素次数。对于长度为 n 的顺序表，假设每个元素被查找的概率相等（均为 1/n），则 ASL=(1+2+…+n)/n=(n+1)/2。
- 插入：
  插入操作是指在顺序表的第 i 个位置插入一个新元素 x，插入后使原表长为 n 的

线性表变为表长为 n+1 的线性表。
插入过程需要将第 i 个位置及之后的所有元素依次向后移动一个位置，以腾出第 i 个位置存放新元素 x。
最坏情况下，需要移动 n 个元素（在第一个位置插入），时间复杂度为 O(n)。
平均移动元素次数为：假设在每个位置插入的概率相等（均为 1/(n+1)），则平均移动次数为 (0+1+2+…+n)/(n+1)=n/2。

- 删除：
删除操作是指删除顺序表中第 i 个元素，删除后使原表长为 n 的线性表变为表长为 n-1 的线性表。
删除过程需要将第 i+1 个位置及之后的所有元素依次向前移动一个位置，以填补被删除元素留下的空缺。
最坏情况下，需要移动 n-1 个元素（删除第一个元素），时间复杂度为 O(n)。
平均移动元素次数为：假设删除每个元素的概率相等（均为 1/n），则平均移动次数为 (0+1+2+…+(n-1))/n=(n-1)/2。
- 线性表链式表示：
线性表的链式表示又称为链式存储结构或链表，它是通过一组任意的存储单元来存储线性表中的元素，元素之间的逻辑关系通过指针来表示。
- 单链表：
单链表是最简单的链表形式，每个节点由数据域和指针域两部分组成。

```
struct Node {
int data; // 数据域
struct Node* next; // 指针域，指向后继节点
};
```

- 数据域：用于存储数据元素的值。
- 指针域：用于存储指向其后继节点的指针（地址）。
单链表通过头指针来标识，头指针指向链表的第一个节点；若链表为空，则头指针为 NULL。例如，一个存储整数的单链表节点可以表示为：
- 基本操作：
- 创建：
单链表的创建通常有头插法和尾插法两种方式。
头插法是从一个空表开始，每次将新节点插入到当前链表的头部（头指针之后）。这种方法创建的链表中，元素的插入顺序与链表中元素的顺序相反。
尾插法是从一个空表开始，设置一个尾指针始终指向当前链表的最后一个节点，每次将新节点插入到尾指针之后，并更新尾指针。这种方法创建的链表中，元素的插入顺序与链表中元素的顺序一致。
- 插入：
在单链表的第 i 个位置插入新元素 x，需要先找到第 i-1 个节点，然后将新节点的指针域指向第 i 个节点，再将第 i-1 个节点的指针域指向新节点。
插入操作的时间复杂度主要取决于查找第 i-1 个节点的时间，为 O(n)。
- 删除：
删除单链表的第 i 个元素，需要先找到第 i-1 个节点，然后将第 i-1 个节点的指针域指向第 i+1 个节点，最后释放第 i 个节点的存储空间。

删除操作的时间复杂度同样主要取决于查找第 i-1 个节点的时间，为 O(n)。

- 查找：
  按值查找：从表头开始，依次比较每个节点的数据域与目标值 x，若找到则返回该节点的指针；若遍历完链表仍未找到，则返回 NULL，时间复杂度为 O(n)。
  按位查找：从表头开始，依次移动指针，直到找到第 i 个节点并返回其指针；若 i 超出链表长度，则返回 NULL，时间复杂度为 O(n)。
- 比较：
- 顺序表与链表：
- 存储方式：顺序表采用连续的存储单元，链表采用任意的存储单元。
- 存取效率：顺序表支持随机存取，按索引访问元素的时间复杂度为 O(1)；链表只能顺序存取，访问元素必须从表头开始依次查找，时间复杂度为 O(n)。
- 插入删除操作：顺序表的插入和删除需要移动大量元素，时间复杂度为 O(n)；链表的插入和删除只需修改指针，时间复杂度主要取决于查找位置的时间，平均为 O(n)，但在已知前驱节点位置时，插入和删除操作可在 O(1) 时间内完成。
- 空间分配：顺序表在静态分配时可能存在存储空间浪费或溢出的问题；链表采用动态分配，存储空间利用率高，且不会出现溢出问题（除非内存不足）。
- 单链表、循环链表和双向链表：
- 单链表：最后一个节点的指针域为 NULL，只能从头指针开始遍历整个链表，无法从后向前遍历。
- 循环链表：最后一个节点的指针域指向头节点，形成一个环形结构。它可以从任意节点开始遍历整个链表，适合用于解决约瑟夫环等问题。
- 双向链表：每个节点除了数据域和指向后继节点的指针外，还增加了一个指向其前驱节点的指针。双向链表可以方便地从一个节点访问其前驱和后继节点，使得插入和删除操作更加灵活，但需要额外的存储空间来存储前驱指针。

第三章 栈和队列

- **栈的定义和特点：**

栈是一种特殊的线性表，其特殊性在于仅允许在表的一端进行插入和删除操作。这一端被称为栈顶，另一端则被称为栈底。
栈的核心特点是遵循“后进先出”（Last In First Out，LIFO）原则，即最后插入栈的元素会最先被删除。
例如，日常生活中的叠放书本就是栈的典型实例：最后放在最上面的书本，往往是最先被取走的；食堂里叠放的餐盘也遵循这一规律，最后放上的餐盘会被先取用。
栈的基本操作包括：入栈（在栈顶插入元素）、出栈（删除栈顶元素）、取栈顶元素（获取栈顶元素的值但不删除）、判断栈空（检查栈是否没有元素）等。

- **栈的存储结构：**
- **顺序栈：**

顺序栈是采用顺序存储结构实现的栈，通常使用一维数组来存储栈中的元素，并通过一个栈顶指针（top）来指示栈顶元素的位置。
假设数组的大小为 MaxSize，栈顶指针 top 的初始值为 -1（表示栈为空）。当元素入栈时，top 的值加 1，然后将元素存入 top 所指的位置；当元素出栈时，先取出 top 所指位置的元素，然后 top 的值减 1。
例如，使用 C 语言定义一个容量为 10 的整型顺序栈：

```
#define MaxSize 10
typedef struct {
int data[MaxSize]; // 存储栈元素的数组
int top; // 栈顶指针，初始值为-1
} SeqStack;
```

顺序栈的操作具有如下特点：

- **入栈时需先判断栈是否已满（top == MaxSize - 1），若已满则无法入栈，称为栈溢出。**
- **出栈和取栈顶元素时需先判断栈是否为空（top == -1），若为空则无法执行相应操作。**
- **所有操作的时间复杂度均为 O (1)，因为只需对栈顶指针进行操作。**
- **链栈：**

链栈是采用链式存储结构实现的栈，通常使用单链表来存储栈中的元素，链表的头指针即为栈顶指针，指向栈顶元素。
链栈的节点结构与单链表类似，包含数据域和指针域，其中指针域指向其前驱节点（即栈中的下一个元素）。
例如，使用 C 语言定义一个存储整型数据的链栈节点：

```
typedef struct StackNode {
int data; // 数据域
struct StackNode* next; // 指针域，指向前驱节点
} StackNode, *LinkStack;
```

链栈的操作具有如下特点：

- 入栈时只需创建新节点，将其指针域指向当前栈顶节点，再将栈顶指针更新为新节点，无需判断栈是否已满（除非内存不足）。
- 出栈时需先判断栈是否为空（栈顶指针为 NULL），若不为空，则取出栈顶节点的数据，将栈顶指针更新为栈顶节点的下一个节点，然后释放原栈顶节点的存储空间。

- 所有操作的时间复杂度均为 O (1)，因为只需对栈顶指针进行操作。
- 链栈的存储空间是动态分配的，相比于顺序栈，更能灵活地适应元素数量的变化。

- 队列的定义和特点：

队列是另一种特殊的线性表，其特殊性在于仅允许在表的一端进行插入操作，在另一端进行删除操作。允许插入的一端称为队尾，允许删除的一端称为队头。
队列的核心特点是遵循“先进先出”（First In First Out，FIFO）原则，即最先插入队列的元素会最先被删除。
例如，日常生活中的排队买票就是队列的典型实例：先排队的人会先买到票离开，后排队的人则需要等待前面的人完成操作。
队列的基本操作包括：入队（在队尾插入元素）、出队（在队头删除元素）、取队头元素（获取队头元素的值但不删除）、判断队空（检查队列是否没有元素）等。

- 队列的存储结构：
- 循环队列：

循环队列是为了解决顺序队列的“假溢出”问题而设计的。顺序队列在元素出队后，队头前的空间无法被再次利用，当队尾指针达到数组末尾时，即使队列中还有空闲空间，也会被认为是栈满，这种现象称为假溢出。
循环队列将存储队列元素的数组视为一个环形结构，当队尾指针达到数组末尾时，若数组头部还有空闲空间，则队尾指针会绕回数组头部。
循环队列通常使用两个指针来标识：队头指针（front）指向队头元素的前一个位置，队尾指针（rear）指向队尾元素。
例如，使用 C 语言定义一个容量为 10 的整型循环队列：

```
#define MaxSize 10
typedef struct {
int data[MaxSize]; // 存储队列元素的数组
int front; // 队头指针，指向队头元素的前一个位置
int rear; // 队尾指针，指向队尾元素
} CirQueue;
```

循环队列的相关操作及判断条件如下：

- 队空条件：front == rear。
- 队满条件：

- 牺牲一个存储单元法：(rear + 1) % MaxSize == front。这种方法通过牺牲一个存储单元来区分队满和队空，当队尾指针的下一个位置是队头指针时，认为队列已满。

- 标志位法：设置一个标志位 flag，当 flag 为 0 时表示队列可能为空，当 flag 为 1 时表示队列可能已满。队空条件为 front == rear 且 flag == 0；队满条件为 front == rear 且 flag == 1。入队成功后将 flag 设为 1，出队成功后将 flag 设为 0。
- 求长度：(rear - front + MaxSize) % MaxSize。
- 入队：先判断队列是否已满，若未满，则将队尾指针后移（(rear + 1) % MaxSize），然后将元素存入队尾指针所指位置（若采用牺牲一个存储单元法，则是先存元素再移指针，具体取决于定义）。
- 出队：先判断队列是否为空，若不为空，则将队头指针后移（(front + 1) % MaxSize），然后取出队头指针所指位置的元素。

循环队列的所有操作时间复杂度均为 O (1)。

- 链队列：

链队列是采用链式存储结构实现的队列，通常使用单链表来存储队列中的元素，包含一个队头指针（front）和一个队尾指针（rear）。队头指针指向队头节点，队尾指针指向队尾节点。

为了操作方便，链队列通常会设置一个头节点，头节点的指针域指向队头节点，队头指针指向头节点。当队列为空时，队头指针和队尾指针都指向头节点。
例如，使用 C 语言定义一个存储整型数据的链队列：

```
typedef struct QNode {
int data; // 数据域
struct QNode* next; // 指针域，指向后继节点
} QNode, *QueuePtr;

typedef struct {
QueuePtr front; // 队头指针，指向头节点
QueuePtr rear; // 队尾指针，指向队尾节点
} LinkQueue;
```

链队列的操作具有如下特点：

- 入队时只需创建新节点，将队尾节点的指针域指向新节点，再将队尾指针更新为新节点，无需判断队列是否已满（除非内存不足）。
- 出队时需先判断队列是否为空（队头指针的指针域为 NULL），若不为空，则获取队头节点（头节点的下一个节点）的数据，将头节点的指针域指向队头节点的下一个节点，若队头节点是队尾节点（即出队后队列为空），则将队尾指针也指向头节点，最后释放原队头节点的存储空间。
- 所有操作的时间复杂度均为 O (1)。
- 链队列的存储空间是动态分配的，能灵活适应元素数量的变化。
- 栈和队列比较：
- 定义：栈是仅允许在一端进行插入和删除操作的线性表；队列是允许在一端插入、另一端删除操作的线性表。

- 操作特点：栈遵循“后进先出”原则；队列遵循“先进先出”原则。
- 存储结构：
- 栈可采用顺序存储（顺序栈）和链式存储（链栈），顺序栈需注意栈溢出问题，链栈则无此问题。
- 队列可采用顺序存储（循环队列）和链式存储（链队列），循环队列需解决队满和队空的判断问题，链队列则无需担心队列溢出。
- 应用场景：
- 栈的典型应用包括表达式求值、函数调用过程中的参数和返回地址保存、括号匹配检查等。例如，在编译原理中，利用栈来实现表达式的语法分析；在递归函数调用中，系统通过栈来保存每一层递归的局部变量和返回地址。
- 队列的典型应用包括操作系统中的进程调度、打印机的任务队列、网络数据的缓冲传输等。例如，多个进程竞争使用打印机时，操作系统会将这些打印任务放入队列，按照“先进先出”的原则依次处理；在网络通信中，数据到达的速度可能快于处理速度，此时会将数据放入队列进行缓冲。

第四章 串、数组和广义表

- **串：**
- **定义：**

串（String）是由零个或多个字符组成的有限序列，又称为字符串。一般记为 S = "$a_1a_2\cdots a_n$"，其中 S 是串名，用双引号（或单引号）括起来的字符序列是串的值，a_i（1≤i≤n）是串中的字符，n 是串的长度。当 n=0 时，称为空串（Empty String），记为 ""。
串中任意个连续的字符组成的子序列称为该串的子串，包含子串的串称为主串。某个字符在串中的位置是指该字符在串中的序号，子串在主串中的位置则以子串的第一个字符在主串中的位置来表示。例如，设主串 S = "abcdef"，子串 T = "cde"，则 T 在 S 中的位置是 3。

串的基本操作与线性表类似，但串的操作通常以整个串或子串为单位进行，如串的比较、连接、求子串等。

- **存储结构：**
- **顺序存储：**

串的顺序存储结构是用一组地址连续的存储单元来存储串中的字符序列，又称为顺序串。可以使用定长数组来实现，即预先定义串的最大长度，当串的实际长度小于最大长度时，用特殊字符（如 '\0'）填充剩余空间。
例如，在 C 语言中，使用字符数组存储串：

```
#define MAXLEN 256
typedef struct {
char ch[MAXLEN]; // 存储串的字符数组
int length; // 串的实际长度
} SString;
```

顺序存储的优点是随机存取方便，缺点是串的长度固定，可能造成存储空间的浪费或溢出。

- **链式存储：**

串的链式存储结构是用链表来存储串中的字符序列，又称为链串。通常每个节点存储一个或多个字符，前者称为单字符节点链串，后者称为块链结构。
单字符节点链串的节点结构如下：

```
typedef struct StringNode {
char data; // 存储一个字符
```

```
struct StringNode* next; // 指向下一个节点
} StringNode, *LinkString;
```

块链结构中，每个节点存储多个字符（如一个块），可以减少指针的数量，提高存储效率。例如，每个节点存储 4 个字符：

```
#define BLOCKSIZE 4
typedef struct Block {
char ch[BLOCKSIZE]; // 存储多个字符
struct Block* next; // 指向下一个块
} Block, *BlockString;
```

链式存储的优点是串的长度可以动态变化，便于插入和删除操作，缺点是存储密度较低（单字符节点链串），且不支持随机存取。

- 模式匹配算法：

模式匹配是指在主串中查找与模式串（子串）相等的子串，并返回其在主串中的位置，是串的核心操作之一。

- BF 算法：

BF（Brute - Force）算法又称暴力匹配算法，其基本思想是：从主串 S 的第一个字符开始，将主串 S 的字符与模式串 T 的字符依次比较。若相等，则继续比较后续字符；若不相等，则回溯到主串 S 的下一个字符，重新与模式串 T 的第一个字符比较，直到找到匹配的子串或遍历完主串。

例如，设主串 S = "ababcabcacbab"，模式串 T = "abcac"，BF 算法的匹配过程如下：

- 初始时，i=0（主串指针），j=0（模式串指针），比较 S [0] 与 T [0]，相等，i=1，j=1。
- 比较 S [1] 与 T [1]，相等，i=2，j=2。
- 比较 S [2] 与 T [2]，不相等，i=1，j=0，重新开始比较。
- 重复上述过程，直到 i=6，j=0 时，依次比较 S [6] 与 T [0]、S [7] 与 T [1]、S [8] 与 T [2]、S [9] 与 T [3]、S [10] 与 T [4]，均相等，匹配成功，返回位置 6。

BF 算法的时间复杂度在最坏情况下为 O (n×m)，其中 n 是主串的长度，m 是模式串的长度，适用于模式串较短的情况。

- KMP 算法：

KMP 算法由 Knuth、Morris 和 Pratt 共同提出，其改进之处在于：当匹配过程中出现字符不相等时，不需要回溯主串指针，而是利用已经得到的部分匹配结果，将模式串向右滑动尽可能远的距离，从而减少比较次数。

KMP 算法的核心是计算模式串的部分匹配表（又称 next 数组），next [j] 表示当模式串中第 j 个字符与主串中相应字符不匹配时，模式串应该回溯到的位置。

next 数组的定义如下：

- next [0] = -1（约定）。
- 对于 j>0，next [j] 是满足 0 < k < j 且 "T [0..k-1] = T [j - k..j - 1]" 的最大 k，若不存在这样的 k，则 next [j] = 0。

例如，模式串 T = "abcac" 的 next 数组为：next [0] = -1，next [1] = 0，next [2] = 0，next [3] = 0，next [4] = 1。

KMP 算法的匹配过程与 BF 算法类似，但当出现不匹配时，主串指针 i 不变，模式串指针

j = next [j]，直到 j = -1 或找到匹配的字符。
KMP 算法的时间复杂度为 O (n + m)，其中 n 是主串的长度，m 是模式串的长度，效率明显高于 BF 算法，尤其适用于主串和模式串都较长的情况。

- 数组：
- 类型定义：

数组是由相同类型的数据元素构成的有序集合，每个元素可以通过一组下标（Index）来唯一确定其在数组中的位置。数组可以是一维的，也可以是多维的。
一维数组可以看作是线性表的推广，其元素之间是一对一的关系。二维数组可以看作是“数组的数组”，即每个元素都是一个一维数组，元素之间存在行和列的关系。例如，一个 m 行 n 列的二维数组 A 可以表示为 A [m][n]，其中 A [i][j] 表示第 i 行第 j 列的元素。
数组一旦定义，其维数和各维的长度就固定不变，因此数组的操作主要是取值和赋值，时间复杂度均为 O (1)。

- 顺序存储：

数组在计算机中只能采用顺序存储结构，即将数组中的元素按某种顺序排列在一组地址连续的存储单元中。常见的存储方式有按行优先和按列优先两种。

- 按行优先存储：

先存储第一行的元素，再存储第二行的元素，依次类推，每行的元素按列下标从小到大的顺序存储。大多数编程语言（如 C、C++、Java）采用按行优先存储方式。
例如，对于二维数组 A [3][4] = {{a_{00}, a_{01}, a_{02}, a_{03}}, {a_{10}, a_{11}, a_{12}, a_{13}}, {a_{20}, a_{21}, a_{22}, a_{23}}}，按行优先存储的顺序为：a_{00}, a_{01}, a_{02}, a_{03}, a_{10}, a_{11}, a_{12}, a_{13}, a_{20}, a_{21}, a_{22}, a_{23}。
对于多维数组，按行优先存储可以推广为：先存储最左边下标变化最慢的维度，再依次存储其他维度。

- 按列优先存储：

先存储第一列的元素，再存储第二列的元素，依次类推，每列的元素按行下标从小到大的顺序存储。FORTRAN 语言采用按列优先存储方式。
例如，上述二维数组 A [3][4] 按列优先存储的顺序为：a_{00}, a_{10}, a_{20}, a_{01}, a_{11}, a_{21}, a_{02}, a_{12}, a_{22}, a_{03}, a_{13}, a_{23}。
对于一个 m 行 n 列的二维数组，若每个元素占用 k 个存储单元，且 a_{00}的存储地址为 LOC (a_{00})，则按行优先存储时，a_{ij}的存储地址为 LOC (a_{ij}) = LOC (a_{00}) + (i×n + j)×k；按列优先存储时，a_{ij}的存储地址为 LOC (a_{ij}) = LOC (a_{00}) + (j×m + i)×k。

- 特殊矩阵的压缩存储：

特殊矩阵是指具有许多相同元素或零元素，且这些元素的分布具有一定规律的矩阵。为了节省存储空间，可以对特殊矩阵进行压缩存储，即只存储那些非零且不重复的元素。

- 对称矩阵：

若一个 n 阶方阵 A 中的元素满足 a_{ij} = a_{ji}（1≤i, j≤n），则称 A 为对称矩阵。
对称矩阵的上三角部分（i <j）和下三角部分（i> j）的元素是对称相等的，因此只需存储下三角部分（包括主对角线）的元素即可。
按行优先顺序存储下三角部分的元素，共有 n (n + 1)/2 个元素。对于元素 a_{ij}（i ≥ j），其在压缩存储数组中的下标为 k = i (i - 1)/2 + j - 1（假设数组下标从 0 开始）。例如，3 阶对称矩阵 A = [[1, 2, 3], [2, 4, 5], [3, 5, 6]]，压缩存储后为 [1, 2, 4, 3, 5, 6]。

- 三角矩阵：

三角矩阵分为上三角矩阵和下三角矩阵。上三角矩阵是指主对角线以下的元素（i > j）均为常数 c（通常为 0）的矩阵；下三角矩阵是指主对角线以上的元素（i < j）均为常数 c 的矩阵。

对于上三角矩阵，只需存储主对角线及以上的元素和常数 c（只需存储一次）。按行优先顺序存储上三角部分的元素，共有 n(n + 1)/2 个元素，加上常数 c，总存储量为 n(n + 1)/2 + 1。对于元素 a_{ij}（i ≤ j），其在压缩存储数组中的下标为 k = (i - 1)(2n - i + 2)/2 + (j - i)。

对于下三角矩阵，存储方式与对称矩阵类似，只需存储主对角线及以下的元素和常数 c，总存储量也为 n(n + 1)/2 + 1。

- 多对角线矩阵：

多对角线矩阵是指矩阵中除了主对角线及其附近几条对角线以外的元素均为零的矩阵，常见的有三对角线矩阵（带状矩阵）。
三对角线矩阵是指对于 n 阶方阵 A，只有当 |i - j| ≤ 1 时，$a_{ij} \neq 0$，其余元素均为零。
三对角线矩阵的非零元素共有 3n - 2 个，按行优先顺序存储这些非零元素。对于元素 a_{ij}（|i - j| ≤ 1），其在压缩存储数组中的下标为 k = 2(i - 1) + (j - i + 1) = 2i + j - 3（假设数组下标从 0 开始）。例如，4 阶三对角线矩阵的非零元素存储顺序为 $a_{11}, a_{12}, a_{21}, a_{22}, a_{23}, a_{32}, a_{33}, a_{34}, a_{43}, a_{44}$。

- 广义表：
- 定义及特点：

广义表（Generalized List）又称列表，是线性表的推广，它是由零个或多个原子（Atom）或子表（Sublist）组成的有限序列。原子是指不可再分割的基本元素，如整数、字符等；子表是指广义表中的元素本身也是一个广义表。
广义表通常记为 LS = ($a_1, a_2, \cdots, a_n$)，其中 LS 是广义表的名称，a_i（1≤i≤n）可以是原子或子表，n 是广义表的长度。当 n=0 时，称为空广义表，记为 ()。
广义表具有以下特点：

- 递归性：广义表可以是其自身的子表，例如 LS = (1, (2, LS))。
- 多层次性：广义表的元素可以是原子，也可以是子表，子表的元素还可以是子表，从而形成多层次结构，例如 LS = (1, (2, (3, 4)), 5)。
- 共享性：一个广义表可以被多个广义表共享，即多个广义表可以包含同一个子表。

- 表头和表尾：

广义表的表头（Head）是指广义表的第一个元素，可以是原子或子表；表尾（Tail）是指广义表中除表头以外的其余元素构成的广义表，一定是一个广义表。
例如：

- 对于广义表 LS = (a, b, c)，表头是 a，表尾是 (b, c)。
- 对于广义表 LS = ((a, b), c)，表头是 (a, b)，表尾是 (c)。
- 对于广义表 LS = ()，表头和表尾均不存在（或为空）。
- 对于广义表 LS = (a)，表头是 a，表尾是 ()。

- 存储结构：

由于广义表的元素可以是原子或子表，且具有递归性和多层次性，因此通常采用链式存储结构。每个节点分为两类：原子节点和表节点。

```
typedef enum {ATOM, LIST} ElemTag; // ATOM 表示原子节点，LIST 表示表节点
typedef struct GLNode {
ElemTag tag; // 标志域
union {
char atom; // 原子节点的数据域
```

```
struct {
struct GLNode* hp; // 表节点的表头指针域
struct GLNode* tp; // 表节点的表尾指针域
} ptr;
} val;
} GLNode, *GList;
```

这种存储结构可以灵活地表示各种广义表，包括空表、原子、子表以及递归表等。

- 原子节点：用于存储原子，包含一个标志域（表示该节点是原子节点）和一个数据域（存储原子的值）。
- 表节点：用于存储子表，包含一个标志域（表示该节点是表节点）、一个表头指针域（指向子表的表头）和一个表尾指针域（指向子表的表尾）。